KI PRAXIS

Die Handbuchreihe für produktives Arbeiten mit künstlicher Intelligenz

BAND 3

Automatisierung und KI-Agenten

Intelligente Arbeitsabläufe entwickeln und automatisieren

Eugen Becker

1. Auflage, November 2026

Impressum und Nutzungshinweise

Reihe KI PRAXIS, Band 3: Automatisierung und KI-Agenten. Intelligente Arbeitsabläufe entwickeln und automatisieren.

Herausgeber und Autor: Eugen Becker.

1. Auflage, November 2026. Alle Rechte vorbehalten. Das Werk einschließlich aller Teile ist urheberrechtlich geschützt. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes bedarf der Zustimmung des Herausgebers.

Hinweis zu den Codebeispielen: Die Programme in diesem Band wurden mit Python 3 in einer Testumgebung ausgeführt. Sie verwenden nur die Standardbibliothek. Die Ausgaben im Buch sind die Ergebnisse dieser Ausführung. Die Beispiele sind Lehrbeispiele und keine fertigen Produktivsysteme. Eine Verwendung mit echten Daten setzt eigene Prüfung, Absicherung und Tests voraus.

Angaben zu Automatisierungsplattformen, KI-Anbietern, Preisen und Oberflächen sind versionsabhängig. Sie ändern sich häufig. Prüfe vor dem Einsatz die aktuelle Dokumentation des jeweiligen Anbieters.

Gesetze und Richtlinien werden in dem Stand zitiert, der zum Zeitpunkt der Drucklegung galt. Maßgeblich ist immer die aktuell gültige Fassung.

Vorwort

Die ersten beiden Bände haben gezeigt, wie du ein Sprachmodell als Gegenüber nutzt. Dieser Band beschreibt den nächsten Schritt: Ein Modell übernimmt nicht nur eine Antwort, sondern wirkt in einem Ablauf mit. Es liest eine Nachricht, ruft eine Schnittstelle auf, bewertet ein Ergebnis und entscheidet, was als Nächstes geschieht.

Das ist nützlich, weil wiederkehrende Arbeit viel Zeit frisst. Es ist aber auch riskant, weil ein Ablauf ohne Aufsicht Fehler in großem Umfang erzeugt. Dieser Band versucht, beides ernst zu nehmen. Er zeigt, wie Automatisierung funktioniert, und er beschreibt die Kontrollen, ohne die sie nicht verantwortbar ist.

Vorausgesetzt werden die Grundregeln aus Band 1 und das Verständnis für Quellen und Prüfung aus Band 2. Neu sind Schnittstellen, strukturierte Ausgaben, Agenten und Protokollierung. Die Codebeispiele sind bewusst klein gehalten, damit du jede Zeile nachvollziehen kannst.

Einleitung und Zielgruppe

Dieser Band richtet sich an Menschen, die wiederkehrende Abläufe vereinfachen möchten, und an alle, die sich mit den Chancen und Risiken von KI-Agenten vertraut machen wollen. Für die Kapitel 1 bis 5 genügt ein konzeptionelles Verständnis. Die Kapitel 3, 4 und 6 enthalten Programmbeispiele, die du mit grundlegenden Python-Kenntnissen nachvollziehen kannst.

Vorausgesetzt werden ein Computer, ein Texteditor und eine installierte Python-Version. Die Programme laufen ohne zusätzliche Pakete. Wenn du noch keine Erfahrung mit Python hast, lies die Beispiele zuerst als Ablauf und führe sie danach aus. Die Ausgaben im Buch zeigen, was du erwarten solltest.

Die Hinweisboxen aus den früheren Bänden kehren wieder. Achtung-Kästen sind in diesem Band besonders wichtig, weil sie Stellen markieren, an denen ein Fehler echte Folgen hat: beim Versand von Nachrichten, beim Löschen von Daten und bei Zugangsdaten.

Lernziele

- Du kannst einen wiederkehrenden Arbeitsablauf in Auslöser, Schritte und Ergebnis zerlegen.
- Du unterscheidest Automatisierungswerkzeuge nach Hosting, Bedienung und Erweiterbarkeit und kannst eine Wahl begründen.
- Du verstehst, wie REST-Schnittstellen, JSON, Authentifizierung und Webhooks zusammenwirken, und kannst einen Abruf mit Zugangsschlüssel auswerten.

- Du übergibst Daten an ein Sprachmodell, prüfst strukturierte Antworten und behandelst Fehler mit Wiederholung und Protokoll.
- Du erklärst, was einen KI-Agenten von einem Chatbot unterscheidet, und kannst Speicher, Planung und Werkzeuge beschreiben.
- Du baust einen kleinen Agenten mit erlaubten Werkzeugen, Schrittgrenze und menschlicher Freigabe.
- Du kannst mehrstufige Abläufe mit Protokoll und Kennzahlen überwachen.

Inhaltsverzeichnis

Kapitel 1: Grundlagen der Automatisierung 9

1.1 Was ist Automatisierung? 9

1.2 Klassische Automatisierung und KI 9

1.3 Ereignisse und Auslöser 10

1.4 Workflows und Prozessketten 10

Kapitel 2: Automatisierungswerkzeuge 12

2.1 Eine Einordnung der Werkzeuge 12

2.2 Unterschiede und Einsatzbereiche 12

2.3 Lokale und Cloud-basierte Automatisierung 13

Kapitel 3: APIs und Schnittstellen 15

3.1 Was ist eine API? 15

3.2 REST-Schnittstellen 15

3.3 HTTP und JSON 15

3.4 Authentifizierung und API-Keys 19

3.5 Webhooks 19

Kapitel 4: KI in Workflows integrieren 22

4.1 Sprachmodelle einbinden 22

4.2 Daten übergeben 22

4.3 Antworten verarbeiten und prüfen 23

4.4 Strukturierte Ausgaben 23

4.5 Fehlerbehandlung 25

Kapitel 5: KI-Agenten verstehen 28

5.1 Definition und Architektur 28

5.2 Agenten und klassische Chatbots 28

5.3 Werkzeuge und Funktionen 29

5.4 Speicher und Kontext 29

5.5 Planung und Ausführung 29

Kapitel 6: Eigene KI-Agenten entwickeln 31

6.1 Agenten planen 31

6.2 Aufgaben und Werkzeuge definieren 31

6.3 Kontrollmechanismen 31

6.4 Menschliche Freigaben 32

6.5 Ein Agent mit Freigabe 32

6.6 Grenzen autonomer Systeme 34

Kapitel 7: Fortgeschrittene Automatisierung 36

7.1 Mehrstufige Workflows 36

7.2 Mehrere Agenten kombinieren 36

7.3 Agentenkommunikation 36

7.4 Protokollierung 37

7.5 Monitoring und Fehlerbehandlung 37

Kapitel 8: Praxisprojekte 40

8.1 Hinweise zu den Projekten 40

8.2 Projekt 1: Automatisierte Dokumentenverarbeitung 40
8.3 Projekt 2: Persönlicher E-Mail-Assistent 41
8.4 Projekt 3: KI-Agent für wiederkehrende Aufgaben 42

Musterlösungen **44**

Anhang: Freigabe-Checkliste für Automatisierungen **47**

Glossar **48**

Quellenverzeichnis **50**

Stichwortverzeichnis **51**

Kapitel 1: Grundlagen der Automatisierung

1.1 Was ist Automatisierung?

Automatisierung bedeutet, dass ein Ablauf ohne deine Mitwirkung in jedem Schritt abläuft. Du legst fest, wann er startet, welche Schritte er durchläuft und was am Ende herauskommt. Ein Beispiel ist eine Regel, die jede eingehende Rechnung in einen Ordner ablegt, den Betrag in eine Tabelle überträgt und dir eine Liste der offenen Zahlungen schickt. Der Ablauf erledigt die Routine, du prüfst das Ergebnis.

Die Frage, ob sich eine Automatisierung lohnt, lässt sich mit einer einfachen Rechnung beantworten. Man multipliziert die Häufigkeit des Vorgangs mit der Zeit, die er pro Durchgang kostet, und zieht den Aufwand für Einrichtung, Pflege und Prüfung ab. Ein Vorgang, der einmal im Jahr zehn Minuten dauert, lohnt sich selten. Ein Vorgang, der jeden Tag zwanzig Minuten kostet und fehleranfällig ist, lohnt sich oft sehr.

1.2 Klassische Automatisierung und KI

Klassische Automatisierung arbeitet mit festen Regeln. Wenn ein Feld diesen Wert hat, geschieht jenes. Sie ist verlässlich, solange die Eingaben vorhersehbar sind, und sie scheitert, sobald eine Eingabe von der erwarteten Form abweicht. Eine Rechnung, deren Betrag an einer anderen Stelle steht als sonst, bringt eine regelbasierte Lösung ins Stocken.

KI-Komponenten können mit solchen Abweichungen umgehen. Ein Sprachmodell kann einen Betrag aus einem ungewohnt gestalteten Dokument herauslesen oder den Inhalt einer Nachricht einordnen. Dafür zahlst du mit geringerer Vorhersagbarkeit. Die beste Lösung

kombiniert deshalb beides: feste Regeln für den Ablauf und ein Modell für die Stellen, an denen Sprache verarbeitet werden muss.

1.3 Ereignisse und Auslöser

Jede Automatisierung beginnt mit einem Auslöser. Das kann ein Zeitpunkt sein, etwa jeden Montag um sieben Uhr. Es kann ein Ereignis in einem anderen System sein, etwa ein neuer Eintrag in einer Tabelle oder eine eingehende Nachricht. Auch ein manueller Start mit einem Klick zählt als Auslöser. Die Wahl des Auslösers entscheidet darüber, wie schnell der Ablauf reagiert und wie oft er unnötig läuft.

1.4 Workflows und Prozessketten

Ein Workflow ist die Folge von Schritten zwischen Auslöser und Ergebnis. Ein Schritt nimmt Daten entgegen, verarbeitet sie und gibt ein Ergebnis weiter. Mehrere Workflows, die aufeinander aufbauen, bilden eine Prozesskette. Je länger die Kette, desto wichtiger wird eine Dokumentation, denn bei einem Fehler muss man den Weg zurückverfolgen können.

Schritt für Schritt: Einen Ablauf beschreiben, bevor du ihn baust

1. Schreibe den Auslöser in einem Satz, zum Beispiel „neue Datei im Eingangsordner“.
2. Liste die Schritte in der Reihenfolge, in der sie ausgeführt werden, mit Eingabe und Ausgabe.
3. Markiere jeden Schritt, der Daten verändert, sendet oder löscht.
4. Notiere für jede Markierung, wer den Schritt prüft und wie der Fehlerfall aussieht.
5. Lege fest, welche Ergebnisse du nach einem Durchlauf erwartest und wie du sie kontrollierst.

Erwartetes Ergebnis: Eine schriftliche Beschreibung, die auch jemand ohne Vorwissen verstehen kann. Fehlt eine Markierung oder eine Prüfung, ist der Ablauf noch nicht bereit für eine Automatisierung.

Typische Fehler

Der häufigste Fehler ist, mit der Umsetzung zu beginnen, bevor der Ablauf beschrieben ist. Das Ergebnis ist ein Werkzeug, das niemand versteht und das niemand korrigieren kann. Ein zweiter Fehler ist das Automatisieren eines schlechten Prozesses. Was manuell ineffizient ist, wird automatisiert nicht besser, sondern schneller falsch.

Kurz gesagt

Eine Automatisierung besteht aus Auslöser, Schritten und Ergebnis. Bevor du etwas baust, beschreibst du den Ablauf, markierst die riskanten Schritte und legst die Prüfung fest.

Übungen

Übung 1.1: Beschreibe einen wiederkehrenden Vorgang aus deinem Alltag mit Auslöser, Schritten und Ergebnis. Markiere jeden Schritt, der Daten verändert.

Übung 1.2: Berechne für einen Vorgang deiner Wahl, ab welcher Häufigkeit sich eine Automatisierung rechnet, wenn Einrichtung und Pflege zusammen zwei Stunden dauern.

Kapitel 2: Automatisierungswerkzeuge

2.1 Eine Einordnung der Werkzeuge

Es gibt zahlreiche Werkzeuge, mit denen sich Abläufe zwischen verschiedenen Diensten verbinden lassen. Drei davon sind im deutschsprachigen Raum weit verbreitet: n8n, Make und Zapier. Sie unterscheiden sich in Herkunft, Hosting-Optionen, Bedienung und Preismodell. Diese Unterschiede ändern sich mit der Zeit, deshalb beschreibt dieses Kapitel die Kriterien für eine Auswahl und nicht den aktuellen Stand einzelner Tarife.

2.2 Unterschiede und Einsatzbereiche

n8n lässt sich auf eigener Infrastruktur betreiben. Das gibt dir mehr Kontrolle über Daten und Kosten, verlangt aber Betriebswissen für Installation, Aktualisierung und Sicherung. Die Lizenz ist keine klassische Open-Source-Lizenz, sondern schränkt bestimmte Nutzungen ein. Lies die aktuellen Lizenzbedingungen vor einem kommerziellen Einsatz.

Make und Zapier sind vor allem als Cloud-Dienste bekannt. Du meldest dich an, verbindest deine Konten und baust Abläufe in einer Benutzeroberfläche. Der Einstieg ist meist schneller, dafür liegen die Daten bei einem Anbieter und die Kosten richten sich nach der Zahl der ausgeführten Schritte oder Aufgaben. Beide bieten große Bibliotheken mit vorgefertigten Verbindungen zu bekannten Diensten.

Kriterium	Cloud-Dienst	Selbst betrieben
Einstieg	Schnell, ohne Serverbetrieb	Erfordert Installation und Pflege

Datenhaltung	Beim Anbieter	Auf eigener Infrastruktur
Kosten	Nutzungsabhängig oder nach Tarif	Hardware und Betriebszeit
Verbindungen	Große Bibliothek vorgefertigter Dienste	Abhängig vom Umfang der Installation

Die Tabelle beschreibt Kriterien, keine feste Eigenschaft einzelner Werkzeuge. Viele Dienste bieten inzwischen beide Varianten an. Prüfe deshalb immer die aktuellen Angaben des Anbieters.

2.3 Lokale und Cloud-basierte Automatisierung

Die Entscheidung zwischen lokal und Cloud hängt von drei Fragen ab. Erstens, welche Daten durch den Ablauf fließen und ob sie vertraulich sind. Zweitens, wie viel Zeit du in den Betrieb investieren kannst. Drittens, ob der Ablauf auf Dienste zugreift, die ohnehin in der Cloud liegen. Ein Ablauf, der nur interne Dateien verschiebt, ist lokal oft sinnvoll. Ein Ablauf, der E-Mails aus einem Cloud-Konto liest, ist in der Cloud oft naheliegender.

Achtung: Zugangsdaten in Werkzeugen

Automatisierungswerkzeuge speichern Zugangsdaten zu deinen Konten, oft mit weitreichenden Rechten. Vergib für jede Verbindung nur die Rechte, die der Ablauf braucht, und prüfe regelmäßig, welche Verbindungen bestehen. Ein Werkzeug mit Zugriff auf dein gesamtes Postfach ist eine schwere Angriffsfläche, wenn es kompromittiert wird.

Typische Fehler

Der häufigste Fehler bei der Werkzeugwahl ist, nach dem Funktionsumfang zu entscheiden statt nach dem konkreten Ablauf. Ein zweiter ist, die Kosten pro Ausführung zu unterschätzen, wenn ein Ablauf häufiger läuft als gedacht. Ein dritter ist, Zugangsdaten für Testzwecke mit vollen Rechten anzulegen und sie später nicht zu entfernen.

Kurz gesagt

Werkzeuge unterscheiden sich in Hosting, Datenhaltung, Kosten und Bedienung. Die richtige Wahl folgt aus dem Ablauf, den du automatisieren willst, und aus den Daten, die er berührt.

Übungen

Übung 2.1: Vergleiche für einen Ablauf deiner Wahl eine Cloud-Lösung und eine selbst betriebene Lösung anhand der Kriterien aus der Tabelle. Begründe deine Wahl.

Übung 2.2: Erstelle eine Liste aller Verbindungen, die du bereits eingerichtet hast, und notiere für jede die erteilten Rechte.

Kapitel 3: APIs und Schnittstellen

3.1 Was ist eine API?

Eine API, eine Anwendungsprogrammierschnittstelle, ist eine festgelegte Art, mit einem Dienst zu sprechen. Sie beschreibt, welche Anfragen möglich sind, welche Daten man übergeben muss und in welcher Form die Antwort zurückkommt. Für dich als Anwender bedeutet das: Ein Werkzeug kann nur mit einem Dienst arbeiten, wenn dieser eine solche Schnittstelle anbietet und du sie nutzen darfst.

3.2 REST-Schnittstellen

REST beschreibt einen Stil für Schnittstellen im Web, den Roy Fielding in seiner Dissertation im Jahr 2000 formuliert hat. Die Grundidee ist, dass Ressourcen über Adressen erreichbar sind, etwa einer Adresse für Aufgaben oder Kunden. Mit HTTP-Methoden legt man fest, was man damit tun will. Lesen erfolgt mit GET, Anlegen mit POST, Ändern und Löschen mit PUT, PATCH und DELETE.

3.3 HTTP und JSON

HTTP ist das Protokoll, über das Anfragen und Antworten übertragen werden. Jede Antwort trägt einen Statuscode. Die Klasse 2xx zeigt Erfolg an, 4xx einen Fehler auf Seiten des Aufrufers, 5xx einen Fehler auf Seiten des Dienstes. Der Statuscode 401 bedeutet beispielsweise, dass die Anmeldung fehlt oder falsch ist.

JSON ist ein Textformat für strukturierte Daten, das mit Schlüssel-Wert-Paaren, Listen, Zahlen, Texten und Wahrheitswerten arbeitet. Es ist für Menschen lesbar und für Programme leicht zu verarbeiten. Die Grundlagen sind in der Spezifikation RFC 8259 festgelegt. Der folgende Abschnitt zeigt eine vollständige Anfrage gegen einen lokalen Testserver, damit du die Mechanik sehen kannst,

ohne auf einen fremden Dienst angewiesen zu sein.

Codebeispiel 3.1: Abruf und Anlegen über eine REST-Schnittstelle

```
import json, threading, urllib.request, urllib.error
from http.server import HTTPServer, BaseHTTPRequestHandler

API_KEY = "test-key-123"
TASKS = [{"id": 1, "titel": "Angebot pruefen", "erledigt": False}]


class Handler(BaseHTTPRequestHandler):
    def _send(self, status, daten):
        body = json.dumps(daten, ensure_ascii=False).encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def _authorized(self):
        return self.headers.get("Authorization") == f"Bearer {API_KEY}"

    def do_GET(self):
        if not self._authorized():
            return self._send(401, {"fehler": "Schluessel ungueltig"})
        if self.path == "/api/tasks":
            return self._send(200, TASKS)
        self._send(404, {"fehler": "Pfad unbekannt"})

    def do_POST(self):
        if not self._authorized():
            return self._send(401, {"fehler": "Schluessel ungueltig"})
        laenge = int(self.headers.get("Content-Length", 0))
        daten = json.loads(self.rfile.read(laenge).decode("utf-8"))
        neu = {"id": len(TASKS) + 1, "titel": daten["titel"],
               "erledigt": False}
        TASKS.append(neu)
        self._send(201, neu)

    def log_message(self, *args):
        pass


server = HTTPServer(("127.0.0.1", 8765), Handler)
threading.Thread(target=server.serve_forever, daemon=True).start()


def anfrage(methode, pfad, daten=None, schluessel=API_KEY):
    url = "http://127.0.0.1:8765" + pfad
    body = json.dumps(daten).encode("utf-8") if daten else None
    req = urllib.request.Request(url, method=methode, data=body,
        headers={"Authorization": f"Bearer {schluessel}",
```

```
                "Content-Type": "application/json"})
    try:
        with urllib.request.urlopen(req) as antwort:
            return antwort.status, json.loads(antwort.read())
    except urllib.error.HTTPError as fehler:
        return fehler.code, json.loads(fehler.read())


def zeige(status, daten):
    if isinstance(daten, list):
        print(status, [t["titel"] for t in daten])
    elif "titel" in daten:
        print(status, daten["id"], daten["titel"])
    else:
        print(status, daten["fehler"])


zeige(*anfrage("GET", "/api/tasks"))
zeige(*anfrage("GET", "/api/tasks", schluessel="falsch"))
zeige(*anfrage("POST", "/api/tasks", {"titel": "Protokoll schreiben"}))
zeige(*anfrage("GET", "/api/tasks"))
server.shutdown()
```

Ausgabe

```
200 ['Angebot pruefen']
401 Schluessel ungueltig
201 2 Protokoll schreiben
200 ['Angebot pruefen', 'Protokoll schreiben']
```

Das Programm startet einen kleinen Server im eigenen Rechner und stellt zwei Adressen bereit: eine zum Lesen von Aufgaben und eine zum Anlegen neuer Aufgaben. Die erste Zeile der Ausgabe zeigt den erfolgreichen Abruf mit dem Statuscode 200. Die zweite Zeile zeigt, dass ein falscher Schlüssel mit 401 abgewiesen wird. Die dritte Zeile legt eine neue Aufgabe an, und die vierte Zeile zeigt, dass sie danach in der Liste steht.

3.4 Authentifizierung und API-Keys

Die Anmeldung an einer Schnittstelle geschieht häufig über einen API-Key, einen langen geheimen Schlüssel, den du mit jeder Anfrage mitschickst. Das Beispiel oben zeigt die übliche Form: Der Schlüssel steht im Kopf der Anfrage, nach dem Wort Bearer. Wer den Schlüssel besitzt, kann im Namen des Kontos handeln. Er ist deshalb wie ein Passwort zu behandeln.

> **Achtung: API-Keys nicht in Programmen ablegen**
>
> Schreibe einen Schlüssel niemals direkt in ein Programm oder eine Datei, die du weitergibst oder in ein Versionsverwaltungssystem legst. Übergib ihn stattdessen über eine Umgebungsvariable oder einen Geheimnisspeicher deines Werkzeugs. Ein versehentlich veröffentlichter Schlüssel sollte sofort beim Anbieter gesperrt und durch einen neuen ersetzt werden.

Im Beispiel steht der Schlüssel zu Lehrzwecken im Quelltext. Das ist ausschließlich für den lokalen Testserver vertretbar. In einem echten Ablauf gehört er in eine Umgebungsvariable.

3.5 Webhooks

Bisher hat der Ablauf immer aktiv nachgefragt. Dieses Verfahren heißt Abruf oder Polling. Bei einem Webhook ist es umgekehrt: Der Dienst ruft deinen Ablauf auf, sobald etwas passiert. Ein Webhook ist also eine Adresse, an die ein Dienst eine Nachricht schickt, etwa bei einem neuen Formularstapel oder einer abgeschlossenen Zahlung. Das spart Anfragen, verlangt aber, dass deine Adresse von außen erreichbar und gegen Missbrauch gesichert ist. Diese Sicherung erfolgt meist über ein gemeinsames Geheimnis, mit dem der Dienst seine Nachricht signiert.

Schritt für Schritt: Eine Schnittstelle sicher erkunden

1. Lies die Dokumentation und notiere die Adresse, die Methode, die benötigten Felder und die Authentifizierung.
2. Frage zuerst einen Lesezugriff ab und prüfe die Form der Antwort mit einem Beispiel.
3. Teste den Fehlerfall mit einem absichtlich falschen Schlüssel und notiere den Statuscode.
4. Lege erst dann einen schreibenden Zugriff an, und zwar in einem Testkonto oder mit einem kennzeichnenden Titel.
5. Entferne den Testeintrag wieder, sobald die Prüfung abgeschlossen ist.

Erwartetes Ergebnis: Eine dokumentierte Erkundung mit Statuscodes für Erfolg und Fehler, ohne dass Produktivdaten verändert wurden.

Typische Fehler

Der häufigste Fehler ist, einen Schreibzugriff zu testen, ohne vorher den Lesezugriff zu verstehen. Ein zweiter ist, den Statuscode zu ignorieren und nur den Inhalt der Antwort zu prüfen. Eine Antwort mit Fehlermeldung kann trotzdem als Text ankommen und wie ein Erfolg aussehen.

Kurz gesagt

Schnittstellen sind festgelegte Wege, mit einem Dienst zu sprechen. Anfragen tragen Methode und Schlüssel, Antworten tragen Statuscode und JSON. Schlüssel gehören in geschützte Ablagen, und Webhooks drehen die Richtung des Aufrufs um.

Übungen

Übung 3.1: Führe das Codebeispiel aus und ändere den Schlüssel in der Anfrage auf einen falschen Wert. Welcher Statuscode erscheint?

Übung 3.2: Erkläre in drei Sätzen den Unterschied zwischen Abruf und Webhook an einem Beispiel aus deinem Alltag.

Kapitel 4: KI in Workflows integrieren

4.1 Sprachmodelle einbinden

Ein Sprachmodell wird in einem Ablauf wie jeder andere Dienst angesprochen. Der Ablauf sendet einen Text mit einer Anweisung, und das Modell antwortet mit einem Text. Die genauen Angaben zu Adresse, Modellname, Parametern und Preisen stehen in der Dokumentation des Anbieters und ändern sich mit der Zeit. Sie gehören deshalb nicht in dieses Buch, sondern in die Konfiguration des Ablaufs.

Die Kernfrage bei der Einbindung lautet: Was genau bekommt das Modell, und was erwartest du zurück? Jeder Ablauf mit Sprachmodell hat eine Eingabe, eine Aufgabe und eine Ausgabe. Diese drei Teile solltest du schriftlich festlegen, bevor du sie implementierst.

4.2 Daten übergeben

Wie Daten übergeben werden, entscheidet über Qualität und Sicherheit. Gib nur die Teile eines Dokuments weiter, die für die Aufgabe nötig sind. Trenne Anweisung und Daten deutlich, etwa durch Überschriften oder Begrenzungszeichen. Vermeide es, unvertrauenswürdige Texte aus fremden Quellen ungeprüft in die Anweisung zu mischen. Ein Text, der im Dokument den Satz „ignoriere alle bisherigen Anweisungen“ enthält, kann ein Modell in die Irre führen. Dieses Risiko behandelt Band 8 ausführlich.

4.3 Antworten verarbeiten und prüfen

Eine Antwort des Modells ist zunächst nur Text. Damit ein Programm sie weiterverarbeiten kann, verlangst du ein festes Format, meist JSON, und prüfst jede Antwort gegen eine Liste von Regeln. Die Prüfung bemerkt fehlende Felder, falsche Typen und Werte außerhalb des erlaubten Bereichs. Sie kann keine inhaltlichen Fehler erkennen, die plausibel aussehen. Dafür braucht es die Prüfung durch einen Menschen an den Stellen, die Folgen haben.

4.4 Strukturierte Ausgaben

Das folgende Beispiel zeigt eine solche Prüfung. Die Antwort des Modells wird simuliert, damit das Beispiel ohne Zugang zu einem Anbieter läuft. In einem echten Ablauf ersetzt du diesen Teil durch den Aufruf der Schnittstelle. Die Prüfung bleibt dieselbe.

Codebeispiel 4.1: Prüfung einer strukturierten Antwort

```
import json

PFLICHTFELDER = {"kategorie": str,
                 "prioritaet": int,
                 "zusammenfassung": str}


def pruefe_antwort(text):
    try:
        daten = json.loads(text)
    except json.JSONDecodeError as fehler:
        return None, f"kein gueltiges JSON: {fehler.msg}"
    if not isinstance(daten, dict):
        return None, "Antwort ist kein Objekt"
    for feld, typ in PFLICHTFELDER.items():
        if feld not in daten:
            return None, f"Pflichtfeld fehlt: {feld}"
        if not isinstance(daten[feld], typ):
            return None, f"falscher Typ bei {feld}"
    if not 1 <= daten["prioritaet"] <= 5:
        return None, "prioritaet muss zwischen 1 und 5 liegen"
    return daten, "ok"


tests = [
    '{"kategorie": "Angebot", "prioritaet": 2, '
    '"zusammenfassung": "Frist pruefen"}',
    'Hier ist die Antwort: {"kategorie": "Angebot"}',
    '{"kategorie": "Angebot", "prioritaet": "hoch", '
    '"zusammenfassung": "x"}',
    '{"kategorie": "Angebot", "prioritaet": 9, '
    '"zusammenfassung": "x"}',
]
for t in tests:
    daten, status = pruefe_antwort(t)
    print(status)
```

Ausgabe

```
ok
kein gueltiges JSON: Expecting value
falscher Typ bei prioritaet
prioritaet muss zwischen 1 und 5 liegen
```

Die vier Testfälle decken typische Abweichungen ab. Der erste Fall ist gültig. Der zweite enthält Text vor dem JSON, weshalb das Auslesen scheitert. Der dritte liefert eine Zeichenkette statt einer Zahl. Der vierte

liegt außerhalb des erlaubten Bereichs. Jede Ablehnung enthält eine Begründung, die im Protokoll landen kann.

4.5 Fehlerbehandlung

Schnittstellen antworten manchmal gar nicht, langsam oder mit Fehlern, die später verschwinden. Eine robuste Fehlerbehandlung umfasst drei Elemente. Erstens eine Zeitgrenze, damit ein Ablauf nicht unbegrenzt wartet. Zweitens eine begrenzte Wiederholung mit wachsender Wartezeit, weil viele Fehler vorübergehend sind. Drittens ein Protokoll, das jeden Versuch festhält. Ohne Protokoll weißt du nach einem Fehler nicht, was passiert ist.

Codebeispiel 4.2: Wiederholung mit Protokoll

```
import json, time

LOG = []

def protokoll(ereignis, **felder):
    eintrag = {"ereignis": ereignis, **felder}
    LOG.append(json.dumps(eintrag, ensure_ascii=False))

def mit_wiederholung(aufruf, versuche=3, wartezeit=0.01):
    for nummer in range(1, versuche + 1):
        try:
            ergebnis = aufruf()
            protokoll("erfolg", versuch=nummer)
            return ergebnis
        except TimeoutError as fehler:
            protokoll("fehler", versuch=nummer, grund=str(fehler))
            if nummer == versuche:
                protokoll("aufgegeben", versuche=versuche)
                raise
            time.sleep(wartezeit * 2 ** (nummer - 1))

zaehler = {"n": 0}
def unzuverlaessig():
    zaehler["n"] += 1
    if zaehler["n"] < 3:
        raise TimeoutError("Modell antwortet nicht")
    return "Antwort"

print(mit_wiederholung(unzuverlaessig))
print("\n".join(LOG))
```

Ausgabe

```
Antwort
{"ereignis": "fehler", "versuch": 1, "grund": "Modell antwortet nicht"}
{"ereignis": "fehler", "versuch": 2, "grund": "Modell antwortet nicht"}
{"ereignis": "erfolg", "versuch": 3}
```

Die Funktion versucht den Aufruf höchstens dreimal. Zwischen den Versuchen wartet sie länger, jeweils doppelt so lange wie vorher. Im Beispiel scheitert der Aufruf zweimal und gelingt beim dritten Versuch. Jeder Versuch steht im Protokoll als eigene Zeile im JSON-Format, das du später auswerten kannst. Band 3, Kapitel 7, greift das Thema Protokollierung wieder auf.

Achtung: Wiederholungen bei schreibenden Schritten

Eine Wiederholung ist unbedenklich, wenn der Schritt nur liest. Bei einem schreibenden Schritt kann eine Wiederholung dieselbe Aktion zweimal ausführen, etwa eine Zahlung oder eine versandte Nachricht. Prüfe vorher, ob der Schritt idempotent ist, also bei mehrfacher Ausführung dasselbe Ergebnis liefert, oder sichere den Vorgang über eine eindeutige Kennung ab.

Typische Fehler

Der häufigste Fehler ist, die Antwort des Modells direkt in den nächsten Schritt zu übergeben, ohne sie zu prüfen. Ein zweiter ist, eine unbegrenzte Wiederholung einzubauen, die bei einem dauerhaften Fehler endlos läuft und Kosten verursacht. Ein dritter ist, Fehler stillschweigend zu verschlucken und den Ablauf als erfolgreich zu melden.

Kurz gesagt

Ein Sprachmodell ist im Ablauf ein Dienst unter anderen. Seine Antworten werden geprüft, bevor sie weiterwirken, Fehler werden mit Zeitgrenze, begrenzter Wiederholung und Protokoll behandelt.

Übungen

Übung 4.1: Erweitere die Prüfung im Codebeispiel um eine Regel, dass die Zusammenfassung höchstens 200 Zeichen lang sein darf. Teste sie mit einer zu langen Eingabe.

Übung 4.2: Ändere die Wiederholung so, dass sie nach dem zweiten Fehler aufgibt. Welche Ausgabe erwartest du im Protokoll?

Kapitel 5: KI-Agenten verstehen

5.1 Definition und Architektur

Ein KI-Agent ist ein System, in dem ein Sprachmodell wiederholt entscheidet, welchen nächsten Schritt es ausführt. Es hat Zugriff auf Werkzeuge, also auf Funktionen wie das Lesen eines Kalenders, das Speichern eines Entwurfs oder das Abfragen einer Schnittstelle. Nach jedem Schritt sieht das Modell das Ergebnis und entscheidet erneut, bis die Aufgabe erledigt ist oder das System abbricht.

Die Architektur besteht aus vier Teilen. Das Modell trifft Entscheidungen. Die Werkzeuge führen Aktionen aus. Der Verlauf hält fest, was bisher geschah. Eine Steuerung wiederholt den Zyklus und legt Grenzen fest. Die Steuerung ist der Teil, den du als Mensch am stärksten beeinflusst, und genau dort liegt die Verantwortung.

5.2 Agenten und klassische Chatbots

Ein Chatbot antwortet auf eine Nachricht. Er liefert Text und verändert in der Regel nichts außerhalb des Gesprächs. Ein Agent handelt. Er ruft Werkzeuge auf, die Daten lesen oder schreiben, und seine Entscheidungen können Folgen außerhalb des Gesprächs haben. Dieser Unterschied ist entscheidend für die Risikobewertung: Ein falscher Satz im Chat ist ärgerlich, eine falsch versandte Nachricht ist oft nicht rückholbar.

5.3 Werkzeuge und Funktionen

Werkzeuge sind die Hände des Agenten. Jedes Werkzeug braucht eine klare Beschreibung, welche Eingaben es erwartet und was es zurückgibt. Das Modell wählt anhand dieser Beschreibung aus. Eine unklare Beschreibung führt zu falschen Entscheidungen. Eine zu große Auswahl ebenfalls, weil das Modell dann häufiger das falsche Werkzeug nimmt. Gib dem Agenten nur die Werkzeuge, die er für die Aufgabe braucht.

5.4 Speicher und Kontext

Ein Agent hat zwei Arten von Gedächtnis. Das Arbeitsgedächtnis ist der Verlauf des laufenden Durchgangs, also die bisherigen Schritte und Ergebnisse. Es ist begrenzt durch das Kontextfenster des Modells. Das Langzeitgedächtnis liegt außerhalb des Modells, etwa in einer Datei oder Datenbank, und wird bei Bedarf geladen. Ein Agent erinnert sich also nicht von selbst, sondern nur über das, was ihm der Ablauf wieder vorlegt.

5.5 Planung und Ausführung

Manche Agenten erstellen vor dem ersten Schritt einen Plan, andere entscheiden Schritt für Schritt. Ein Plan macht das Verhalten nachvollziehbarer und lässt sich prüfen, bevor etwas geschieht. Er kann aber veralten, wenn sich die Lage ändert. Für wenig riskante Aufgaben reicht meist die schrittweise Entscheidung. Für Aufgaben mit Folgen lohnt sich ein Plan, den ein Mensch freigibt, bevor die Ausführung beginnt.

Schritt für Schritt: Prüfen, ob eine Aufgabe ein Agent braucht

1. Beschreibe die Aufgabe und die Schritte, die ein Mensch heute ausführt.
2. Prüfe, ob die Schritte festen Regeln folgen. Dann genügt meist eine klassische Automatisierung.
3. Prüfe, ob die Schritte sprachliche Beurteilung oder wechselnde Entscheidungen verlangen.
4. Liste alle Werkzeuge auf, die nötig sind, und markiere jene, die Daten verändern oder senden.
5. Entscheide, welche Markierungen eine menschliche Freigabe brauchen.

Erwartetes Ergebnis: Eine Entscheidung, ob ein Agent nötig ist, mit einer Liste der Werkzeuge und der Freigabepunkte.

Typische Fehler

Der häufigste Fehler ist, einen Agenten für eine Aufgabe einzusetzen, die eine einfache Regel besser löst. Das macht den Ablauf teurer, langsamer und schwerer zu prüfen. Ein zweiter Fehler ist, einem Agenten zu viele Werkzeuge zu geben, aus Bequemlichkeit.

Kurz gesagt

Ein Agent ist ein Modell in einer Schleife mit Werkzeugen. Der Unterschied zum Chatbot liegt im Handeln, und deshalb liegt die Verantwortung bei der Steuerung, den Werkzeugen und den Freigaben.

Übungen

Übung 5.1: Nenne für eine Aufgabe deiner Wahl die Werkzeuge, die ein Agent bräuchte, und markiere jene, die Daten verändern.

Übung 5.2: Entscheide für drei Aufgaben aus deinem Alltag, ob eine klassische Regel oder ein Agent sinnvoller ist. Begründe jede Entscheidung.

Kapitel 6: Eigene KI-Agenten entwickeln

6.1 Agenten planen

Bevor du einen Agenten baust, schreibst du auf, was er erreichen soll, welche Werkzeuge er nutzen darf und wann er anhalten muss. Die Frage, wann er anhält, ist genauso wichtig wie die Frage, was er tut. Ein Agent ohne klare Endbedingung läuft im schlimmsten Fall ohne Ende oder erledigt Dinge, die niemand wollte.

6.2 Aufgaben und Werkzeuge definieren

Eine Aufgabe für einen Agenten beschreibst du so, dass ein Ergebnis überprüfbar ist. „Bereite die Antwort auf Kundenanfragen vor“ ist zu weit. „Lies die Termine der nächsten Woche und speichere einen Entwurf mit Bestätigung der Termine“ ist prüfbar. Jedes Werkzeug bekommt einen Namen, eine Beschreibung und die Angabe, ob es eine Freigabe braucht.

6.3 Kontrollmechanismen

Ein Agent braucht mehrere Kontrollen, die unabhängig voneinander wirken. Eine Liste erlaubter Werkzeuge verhindert, dass das Modell etwas aufruft, das nicht vorgesehen ist. Eine Schrittgrenze begrenzt den Aufwand und verhindert Endlosschleifen. Eine Freigabe hält riskante Aktionen an, bis ein Mensch zustimmt. Ein Protokoll hält fest, was passiert ist. Keine dieser Kontrollen ist vollständig, aber zusammen bilden sie eine tragfähige Grundlage.

6.4 Menschliche Freigaben

Die Freigabe durch einen Menschen ist die wichtigste Kontrolle bei Aktionen mit Folgen. Sie sollte vor der Ausführung erfolgen und die konkrete Aktion zeigen, nicht nur die allgemeine Absicht. Eine Freigabe, die „Der Agent möchte weitermachen“ fragt, ist wertlos, weil niemand weiß, was danach geschieht.

6.5 Ein Agent mit Freigabe

Das folgende Beispiel zeigt einen Agenten mit drei Werkzeugen. Zwei davon dürfen ohne Freigabe laufen, eines braucht sie. Das Modell ist wieder durch eine feste Abfolge simuliert, damit das Beispiel ohne Anbieterzugang läuft. Das Modell versucht unter anderem, ein nicht erlaubtes Werkzeug aufzurufen, und den Versand einer Nachricht ohne Freigabe.

Codebeispiel 6.1: Agent mit Werkzeugliste, Schrittgrenze und Freigabe

```
import json

WERKZEUGE = {
    "termine_lesen": {
        "braucht_freigabe": False,
        "funktion": lambda args: ["Mo 9:00 Teamrunde"]},
    "entwurf_speichern": {
        "braucht_freigabe": False,
        "funktion": lambda args: "Entwurf gespeichert"},
    "mail_senden": {
        "braucht_freigabe": True,
        "funktion": lambda args: "Mail gesendet"},
}


def agent(aufgabe, modell, freigabe, max_schritte=5):
    verlauf = [f"Aufgabe: {aufgabe}"]
    for schritt in range(1, max_schritte + 1):
        entscheidung = json.loads(modell(verlauf))
        if "antwort" in entscheidung:
            print(f"Schritt {schritt}: fertig -> "
                  f"{entscheidung['antwort']}")
            return entscheidung["antwort"]
        name = entscheidung["werkzeug"]
        if name not in WERKZEUGE:
            verlauf.append(f"Fehler: {name} nicht erlaubt")
            print(f"Schritt {schritt}: abgelehnt, "
                  f"{name} ist nicht erlaubt")
            continue
        werkzeug = WERKZEUGE[name]
        if werkzeug["braucht_freigabe"] and not freigabe(
                name, entscheidung["args"]):
            verlauf.append(f"Fehler: Freigabe fuer {name} fehlt")
            print(f"Schritt {schritt}: {name} ohne Freigabe "
                  f"nicht ausgefuehrt")
            continue
        ergebnis = werkzeug["funktion"](entscheidung["args"])
        verlauf.append(f"Ergebnis {name}: {ergebnis}")
        print(f"Schritt {schritt}: {name} ausgefuehrt -> {ergebnis}")
    raise RuntimeError("Maximale Schrittzahl erreicht")


skript = iter([
    {"werkzeug": "termine_lesen", "args": {}},
    {"werkzeug": "loesche_alles", "args": {}},
    {"werkzeug": "mail_senden", "args": {"an": "kunde@example.org"}},
    {"werkzeug": "entwurf_speichern",
     "args": {"text": "Danke fuer das Treffen."}},
```

```
    {"antwort": "Entwurf liegt bereit, Versand wartet auf Freigabe."},
])
modell = lambda verlauf: json.dumps(next(skript))

agent("Termine pruefen und Nachfass-Mail vorbereiten", modell,
      freigabe=lambda name, args: False)
```

Ausgabe

```
Schritt 1: termine_lesen ausgefuehrt -> ['Mo 9:00 Teamrunde']
Schritt 2: abgelehnt, loesche_alles ist nicht erlaubt
Schritt 3: mail_senden ohne Freigabe nicht ausgefuehrt
Schritt 4: entwurf_speichern ausgefuehrt -> Entwurf gespeichert
Schritt 5: fertig -> Entwurf liegt bereit, Versand wartet auf Freigabe.
```

Die Ausgabe zeigt die fünf Schritte. Das Werkzeug zum Lesen der Termine läuft. Das unerlaubte Werkzeug wird abgelehnt. Der Versand wird ohne Freigabe nicht ausgeführt, obwohl das Modell ihn verlangt. Das Speichern des Entwurfs ist erlaubt. Am Ende steht eine Antwort, die den Stand korrekt beschreibt: Der Entwurf liegt bereit, der Versand wartet auf Freigabe.

Die Freigabe im Beispiel lehnt immer ab. In einem realen Ablauf würde sie dem Menschen die konkrete Mail zur Entscheidung zeigen. Die Schrittgrenze liegt bei fünf. Sie ist im Code als Parameter angelegt und lässt sich für andere Aufgaben anpassen.

6.6 Grenzen autonomer Systeme

Agenten machen Fehler auf eine Weise, die man nicht vollständig vorhersehen kann. Sie können Werkzeuge falsch wählen, Ergebnisse falsch deuten oder auf Texte in fremden Quellen hereinfallen, die Anweisungen enthalten. Auch die Kosten sind schwer abzuschätzen, wenn die Zahl der Schritte variiert. Die sicherste Haltung ist, Agenten nur dort einzusetzen, wo ein Fehler schnell auffällt und sich ohne großen Schaden korrigieren lässt.

Achtung: Agenten mit Schreibrechten

Ein Agent, der Daten löschen, Geld überweisen oder Nachrichten im Namen einer Person versenden kann, braucht eine Freigabe für jede dieser Aktionen. Gewähre keine Schreibrechte, die du nicht überwachst. Ein Agent mit weitreichenden Rechten ist ein Risiko für dich und für alle, deren Daten er berührt.

Typische Fehler

Der häufigste Fehler ist, die Kontrollen nur im Kopf zu haben und nicht im Code. Ein zweiter ist, die Freigabe mit einem allgemeinen Häkchen zu erledigen, das niemand liest. Ein dritter ist, die Schrittgrenze zu hoch anzusetzen, damit der Agent „nicht abbricht“.

Kurz gesagt

Ein Agent braucht erlaubte Werkzeuge, eine Schrittgrenze, Freigaben für wirksame Aktionen und ein Protokoll. Diese Kontrollen gehören in die Steuerung, nicht in die Hoffnung auf ein gutes Modell.

Übungen

Übung 6.1: Ändere das Beispiel so, dass der Agent nach jeder Aktion ein Protokoll schreibt. Welche Felder würdest du festhalten?

Übung 6.2: Lege für einen Agenten deiner Wahl eine Liste der Werkzeuge mit Freigabepflicht an. Begründe bei jedem Werkzeug, warum die Freigabe nötig oder nicht nötig ist.

Kapitel 7: Fortgeschrittene Automatisierung

7.1 Mehrstufige Workflows

Ein mehrstufiger Workflow verbindet mehrere Abläufe, bei denen das Ergebnis des einen die Eingabe des nächsten ist. Ein Beispiel ist die Verarbeitung eingehender Dokumente: Erfassen, Prüfen, Ablegen, Benachrichtigen. Jede Stufe sollte ein klar umrissenes Ergebnis haben und einzeln prüfbar sein. Eine Stufe, die gleichzeitig erfasst und entscheidet, lässt sich schwer testen.

7.2 Mehrere Agenten kombinieren

Manche Aufgaben werden übersichtlicher, wenn mehrere Agenten mit getrennten Rollen zusammenarbeiten. Ein Agent sucht Informationen, ein zweiter bewertet sie, ein dritter formuliert das Ergebnis. Die Trennung hat einen Nutzen: Der Prüfagent kann die Arbeit des Rechercheagenten kontrollieren, ohne dessen Annahmen zu übernehmen. Sie hat aber auch einen Preis, denn jede zusätzliche Rolle erhöht Aufwand, Kosten und Fehlerquellen.

7.3 Agentenkommunikation

Agenten sollten untereinander in festen, strukturierten Nachrichten sprechen, etwa in JSON mit Feldern für Auftrag, Ergebnis, Quellen und Status. Freier Text erschwert die Prüfung, weil jeder Empfänger ihn anders deuten kann. Eine Nachricht, die ein Feld für Quellen enthält, zwingt den Absender zu einer Angabe, die der Empfänger prüfen kann.

7.4 Protokollierung

Ein Protokoll ist die Grundlage jeder Fehleranalyse. Ein gutes Protokoll schreibt jedes Ereignis als eigene Zeile mit Zeitpunkt, Art des Ereignisses und den relevanten Feldern. Das Format JSON Lines, bei dem jede Zeile ein eigenes JSON-Objekt ist, eignet sich dafür gut, weil sich die Zeilen mit einfachen Werkzeugen durchsuchen und auswerten lassen. Das Beispiel aus Kapitel 4 erzeugt bereits solche Zeilen.

Protokolle enthalten oft personenbezogene Daten, etwa Namen in Nachrichten oder Inhalte von Dokumenten. Lege fest, welche Felder protokolliert werden, und speichere nur das, was du zur Fehlersuche wirklich brauchst. Lösche Protokolle nach einer festgelegten Frist.

7.5 Monitoring und Fehlerbehandlung

Monitoring bedeutet, regelmäßig zu prüfen, ob ein Ablauf noch wie erwartet arbeitet. Vier Kennzahlen geben einen guten ersten Überblick: die Zahl der Durchläufe, die Fehlerquote, die durchschnittliche Dauer und die Zahl der abgelehnten Freigaben. Eine steigende Fehlerquote oder eine plötzlich abweichende Dauer ist ein Signal, den Ablauf zu prüfen, bevor sich der Fehler ausbreitet.

Schritt für Schritt: Einen Ablauf mit Kennzahlen überwachen

1. Lege fest, welche vier Kennzahlen du verfolgst und welche Werte als auffällig gelten.
2. Schreibe für jeden Durchlauf eine Protokollzeile mit Zeitpunkt, Ergebnis und Dauer.
3. Werte das Protokoll einmal pro Woche aus und notiere die Kennzahlen in einer Tabelle.
4. Prüfe bei jeder Auffälligkeit die letzten fehlgeschlagenen Durchläufe am Protokoll.
5. Passe Wiederholung, Zeitgrenze oder Prüfregeln an, und dokumentiere die Änderung.

Erwartetes Ergebnis: Eine wöchentliche Übersicht, die Auffälligkeiten früh sichtbar macht und jede Änderung am Ablauf begründet.

Typische Fehler

Der häufigste Fehler ist, Protokolle zu schreiben, aber nie auszuwerten. Ein zweiter ist, nur auf Abbrüche zu achten und langsame oder fehlerhafte, aber nicht abgebrochene Durchläufe zu übersehen. Ein dritter ist, Agenten zu vervielfachen, ohne ihre Schnittstellen zu klären.

Kurz gesagt

Fortgeschrittene Automatisierung verlangt klar getrennte Stufen, strukturierte Nachrichten und Protokolle, die man tatsächlich liest. Kennzahlen zeigen, ob der Ablauf noch trägt.

Übungen

Übung 7.1: Entwirf für einen mehrstufigen Ablauf die Protokollzeile für jede Stufe. Welche Felder brauchst du zur Fehlersuche?

Übung 7.2: Lege für deinen Ablauf eine Kennzahl und eine Schwelle fest, bei deren Überschreitung du den Ablauf prüfst. Begründe die

Schwelle.

Kapitel 8: Praxisprojekte

8.1 Hinweise zu den Projekten

Die drei Projekte bauen auf den Kapiteln 1 bis 7 auf. Sie sind so angelegt, dass du sie mit einem Texteditor und Python ausführen kannst, ohne echte Konten anzubinden. Bevor du einen Ablauf mit echten Daten betreibst, teste ihn mit Beispieldaten und prüfe die Freigaben. Ein Projekt gilt als abgeschlossen, wenn das erwartete Ergebnis vorliegt und du jeden Fehlerfall einmal absichtlich ausgelöst hast.

8.2 Projekt 1: Automatisierte Dokumentenverarbeitung

Voraussetzungen und Werkzeuge

1. Python 3 auf deinem Rechner, ohne zusätzliche Pakete.
2. Ein Ordner für eingehende Dokumente und ein Ordner für abgelegte Dokumente.
3. Eine Textdatei mit mindestens fünf Beispieldokumenten, von denen einige fehlerhaft sind.

Hinweis: Der Ablauf verarbeitet nur Beispieldaten. Er dient zum Üben der Kette aus Erfassen, Prüfen und Ablegen.

Durchführung in fünf Schritten

1. Lege die Ordner für eingehende und abgelegte Dokumente an und lege fünf Beispieldokumente hinein.

2. Schreibe einen Ablauf, der jede Datei liest und ihren Inhalt in ein festes JSON-Format überführt.

3. Prüfe jedes Ergebnis mit einer Validierung wie in Codebeispiel 4.1 und lehne unvollständige Dateien ab.

4. Lege gültige Ergebnisse im Zielordner ab und schreibe für jede Datei eine Protokollzeile.

5. Löse einen Fehlerfall absichtlich aus, etwa mit einer fehlenden Angabe, und prüfe, ob das Protokoll den Grund nennt.

Erwartetes Ergebnis: Ein Ablauf, der gültige Dokumente ablegt, ungültige ablehnt und jeden Fall im Protokoll festhält.

Fehlerbehebung: Werden alle Dateien abgelehnt, ist die Prüfregel zu streng oder das Format der Eingabe nicht einheitlich. Fehlen Einträge im Protokoll, ist die Protokollierung nicht an allen Stellen aufgerufen. Sicherheitshinweis: Verwende nur Beispieldaten, bis der Ablauf vollständig getestet ist.

8.3 Projekt 2: Persönlicher E-Mail-Assistent

Dieses Projekt baut einen Assistenten, der Nachrichten vorbereitet, aber nie selbst sendet. Er liest eingehende Nachrichten aus einem Ordner, erstellt zu jeder einen Antwortentwurf und legt ihn zur Prüfung ab. Der Versand bleibt eine manuelle Handlung. Du lernst dabei die wichtigste Regel für persönliche Automatisierung: Der Assistent schreibt Entwürfe, du entscheidest über den Versand.

Durchführung in fünf Schritten

1. Lege einen Ordner mit fünf Beispielnachrichten als Textdateien an.
2. Schreibe einen Ablauf, der jede Nachricht liest und einen Entwurf im Entwurfsordner ablegt.
3. Lass den Ablauf die Entwürfe mit einer Kennzeichnung versehen, die den Status Ungeprüft trägt.
4. Prüfe zwei Entwürfe von Hand, korrigiere sie und ändere den Status auf Freigegeben.
5. Vergewissere dich, dass der Ablauf keine Funktion zum Versenden enthält, und dokumentiere das im Protokoll.

Erwartetes Ergebnis: Ein Entwurfsordner, in dem jeder Entwurf einen Prüfstatus trägt und der Versand bewusst bei dir liegt.

Fehlerbehebung: Entwürfe, die zu allgemein klingen, brauchen eine genauere Anweisung mit Tonfall und Länge. Sicherheitshinweis: Nachrichten enthalten oft persönliche Daten Dritter. Verarbeite nur Nachrichten, bei denen dies für dich in Ordnung ist, und prüfe die Nutzungsbedingungen des genutzten Dienstes.

8.4 Projekt 3: KI-Agent für wiederkehrende Aufgaben

Im dritten Projekt baust du einen Agenten für eine wiederkehrende Aufgabe, etwa das wöchentliche Zusammenstellen einer Übersicht aus mehreren Dateien. Der Agent darf nur lesen und einen Bericht speichern. Eine Freigabe ist für das Speichern nicht nötig, weil es nur eine lokale Datei erzeugt. Sobald du weitere Werkzeuge hinzufügst, die Daten senden oder ändern, gilt Kapitel 6.

Durchführung in fünf Schritten

1. Beschreibe die Aufgabe, ihre Eingaben, das gewünschte Ergebnis und die erlaubten Werkzeuge.
2. Übertrage das Beispiel aus Codebeispiel 6.1 und ersetze die Werkzeuge durch Lese- und Speicherfunktionen für deine Dateien.
3. Setze eine Schrittgrenze, die zu deiner Aufgabe passt, und begründe den Wert.
4. Lass den Agenten einmal laufen und prüfe jeden Schritt im Protokoll.
5. Wiederhole den Lauf mit einem absichtlich fehlerhaften Werkzeugaufruf und prüfe, ob der Agent sauber abbricht.

Erwartetes Ergebnis: Ein Agent, der die wöchentliche Aufgabe erledigt, nur erlaubte Werkzeuge nutzt und jeden Schritt protokolliert.

Fehlerbehebung: Erreicht der Agent die Schrittgrenze, ist die Aufgabe zu groß oder die Werkzeugbeschreibung unklar. Teile die Aufgabe in zwei Agenten oder schärfe die Beschreibung. Sicherheitshinweis: Gib dem Agenten keinen Schreibzugriff auf Ordner, die du nicht ersetzen kannst.

Kurz gesagt

Die drei Projekte zeigen, dass verlässliche Automatisierung mit kleinen, geprüften Schritten beginnt. Der E-Mail-Assistent schreibt Entwürfe und sendet nicht, der Agent darf nur lesen, und die Dokumentenverarbeitung lehnt ungültige Eingaben ab.

Übungen

Übung 8.1: Führe das erste Projekt mit zehn Beispieldokumenten durch und notiere, wie viele abgelehnt wurden und warum.

Übung 8.2: Erweitere den E-Mail-Assistenten um ein Feld, das die Quelle jeder Aussage im Entwurf nennt. Wie prüfst du diese Angabe?

Musterlösungen

Die folgenden Lösungen sind Vorschläge. Bei den Übungen gibt es keine einzig richtige Antwort, entscheidend ist die Begründung und die Prüfung am eigenen Material.

Kapitel 1

Lösung 1.1: Eine vollständige Beschreibung nennt den Auslöser, jeden Schritt mit Eingabe und Ausgabe und die Markierung für riskante Schritte. Fehlt eine Angabe zur Prüfung, ist der Ablauf noch nicht fertig.

Lösung 1.2: Beispiel: Zwei Stunden Aufwand, dass ein Vorgang bei zehn Minuten Ersparnis pro Durchlauf sich nach etwa zwölf Durchläufen rechnet. Die Rechnung ist eine Faustregel und berücksichtigt keine Fehlerkosten.

Kapitel 2

Lösung 2.1: Ein gut begründeter Vergleich nennt die Kriterien Datenhaltung, Kosten und Betriebsaufwand und entscheidet anhand der Daten, die der Ablauf berührt.

Lösung 2.2: Die Liste sollte jede Verbindung mit den erteilten Rechten nennen. Verbindungen mit mehr Rechten, als der Ablauf braucht, sind zu reduzieren.

Kapitel 3

Lösung 3.1: Mit falschem Schlüssel antwortet der Testserver mit dem Statuscode 401. Die Antwort enthält die Fehlermeldung als JSON.

Lösung 3.2: Beim Abruf fragt der Ablauf regelmäßig nach. Beim Webhook meldet der Dienst das Ereignis selbst. Ein Beispiel für einen

Webhook ist eine Benachrichtigung bei einer neuen Bestellung.

Kapitel 4

Lösung 4.1: Die Regel lässt sich mit einer zusätzlichen Längenprüfung ergänzen. Eine zu lange Eingabe sollte abgelehnt werden, mit einer Begründung im Protokoll.

Lösung 4.2: Nach dem zweiten Fehler sollte das Protokoll zwei Fehlerzeilen und eine Aufgabenzeile zeigen. Die Ausgabe ist dann ein Fehler, nicht ein Erfolg.

Kapitel 5

Lösung 5.1: Die Werkzeuge hängen von der Aufgabe ab. Ein Werkzeug, das Daten verändert oder sendet, ist zu markieren und braucht eine Freigabe.

Lösung 5.2: Festen Regeln folgende Vorgänge sind meist mit einer klassischen Automatisierung besser zu lösen. Ein Agent lohnt sich für Aufgaben mit Beurteilung und wechselnden Schritten.

Kapitel 6

Lösung 6.1: Ein sinnvolles Protokollfeld enthält Zeitpunkt, Schritt, Werkzeug, Ergebnis und Freigabestatus. Die Felder sollten keine vollständigen personenbezogenen Inhalte enthalten.

Lösung 6.2: Freigabepflicht besteht bei allen Aktionen, die Daten verändern oder nach außen senden. Lesende Werkzeuge brauchen in der Regel keine Freigabe.

Kapitel 7

Lösung 7.1: Zur Fehlersuche braucht die Protokollzeile den Zeitpunkt, die Stufe, den Status und die Ursache eines Fehlers. Mehr Felder erhöhen den Datenschutzaufwand.

Lösung 7.2: Eine Schwelle ist begründet, wenn sie sich aus dem normalen Verhalten des Ablaufs ergibt, etwa dem Durchschnitt der letzten vier Wochen.

Kapitel 8

Lösung 8.1: Ungültige Dokumente sollten mit einer konkreten Begründung abgelehnt werden. Eine pauschale Ablehnung ist ein Hinweis auf eine zu strenge oder unklare Prüfregel.

Lösung 8.2: Die Quelle lässt sich nur prüfen, wenn sie auf ein konkretes Dokument und eine Stelle verweist. Prüfe die Angabe stichprobenartig am Original.

Anhang: Freigabe-Checkliste für Automatisierungen

Gehe diese Checkliste durch, bevor ein Ablauf im Alltag Daten liest, schreibt oder versendet.

- Auslöser: Ist eindeutig festgelegt, was den Ablauf startet?
- Schritte: Ist jeder Schritt mit Eingabe und Ausgabe beschrieben?
- Rechte: Hat jede Verbindung nur die Rechte, die sie braucht?
- Zugangsdaten: Liegen Schlüssel in einer geschützten Ablage und nicht im Quelltext?
- Freigaben: Ist jede Aktion, die Daten verändert oder sendet, mit einer Freigabe versehen?
- Schrittgrenze: Ist eine Obergrenze gesetzt und begründet?
- Prüfung: Werden Antworten vor der Weiterverarbeitung gegen feste Regeln geprüft?
- Wiederholung: Ist klar, ob ein schreibender Schritt mehrfach ausgeführt werden darf?
- Protokoll: Schreibt der Ablauf für jeden Durchlauf eine nachvollziehbare Zeile?
- Test: Wurde der Fehlerfall mit Beispieldaten einmal absichtlich ausgelöst?

Glossar

Agent: System, in dem ein Sprachmodell in einer Schleife entscheidet, welche Werkzeuge es nutzt, bis eine Aufgabe erledigt ist.

API: Anwendungsprogrammierschnittstelle. Festgelegte Art, Anfragen an einen Dienst zu stellen und Antworten zu empfangen.

API-Key: Geheimer Schlüssel, mit dem sich ein Programm an einer Schnittstelle anmeldet. Er ist wie ein Passwort zu behandeln.

Auslöser: Ereignis oder Zeitpunkt, der einen Ablauf startet.

Allowlist: Liste der ausdrücklich erlaubten Werkzeuge oder Aktionen. Alles andere ist gesperrt.

Freigabe: Zustimmung eines Menschen vor einer Aktion mit Folgen. Man spricht auch von Human in the Loop.

Idempotenz: Eigenschaft eines Schritts, bei mehrfacher Ausführung dasselbe Ergebnis zu liefern.

JSON: Textformat für strukturierte Daten aus Schlüssel-Wert-Paaren, Listen und einfachen Werten.

JSON Lines: Format, bei dem jede Zeile einer Datei ein eigenes JSON-Objekt ist. Geeignet für Protokolle.

Monitoring: Regelmäßige Prüfung, ob ein Ablauf wie erwartet arbeitet, meist anhand von Kennzahlen.

REST: Stil für Schnittstellen im Web, bei dem Ressourcen über Adressen und HTTP-Methoden angesprochen werden.

Retry: Wiederholung eines fehlgeschlagenen Aufrufs, oft mit wachsender Wartezeit.

Schrittgrenze: Obergrenze für die Zahl der Schritte eines Agenten, die Endlosschleifen verhindert.

Statuscode: Dreistellige Zahl in einer HTTP-Antwort, die Erfolg oder Fehler anzeigt.

Timeout: Zeitgrenze, nach der ein Aufruf abgebrochen wird, wenn keine Antwort kommt.

Webhook: Adresse, an die ein Dienst eine Nachricht sendet, sobald ein Ereignis eintritt.

Werkzeug: Funktion, die ein Agent aufrufen kann, etwa das Lesen von Terminen oder das Speichern eines Entwurfs.

Workflow: Folge von Schritten zwischen Auslöser und Ergebnis, die eine Aufgabe automatisiert erledigt.

Quellenverzeichnis

Die folgenden Quellen bilden die Grundlage für die rechtlichen und methodischen Angaben dieses Bandes. Bei Gesetzen gilt stets die aktuell geltende Fassung.

Fielding, Roy T.: Architectural Styles and the Design of Network-based Software Architectures. Dissertation, University of California, Irvine, 2000.

Internet Engineering Task Force: RFC 9110, HTTP Semantics. Juni 2022.

Internet Engineering Task Force: RFC 8259, The JavaScript Object Notation (JSON) Data Interchange Format. Dezember 2017.

Python Software Foundation: Dokumentation der Module json und urllib. Standardbibliothek der jeweils eingesetzten Python-Version.

OWASP: OWASP Top 10 for Large Language Model Applications. Projektdokumentation, Angaben zu Risiken bei Sprachmodell-Anwendungen. Aktuelle Fassung prüfen.

Verordnung (EU) 2024/1689 des Europäischen Parlaments und des Rates über künstliche Intelligenz (KI-Verordnung), Amtsblatt der Europäischen Union, 2024.

Dokumentation der jeweiligen Automatisierungswerkzeuge und KI-Anbieter zu Preisen, Schnittstellen und Datenhaltung. Stand bei jeder Nutzung selbst prüfen.

Stichwortverzeichnis

Eine Auswahl wichtiger Begriffe mit Verweisen auf die Kapitel. Die Angaben beziehen sich auf die Kapitelnummern dieses Bandes.

Begriff	Kapitel
Agent	5, 6, 8
API	3
API-Key	3
Auslöser	1
Freigabe	5, 6, 8
Idempotenz	4
JSON	3, 4
Monitoring	7
Protokollierung	4, 7
REST	3
Retry	4
Schrittgrenze	6
Statuscode	3
Webhook	3
Werkzeug	5, 6
Workflow	1, 7

KI PRAXIS

Band 3 zeigt, wie wiederkehrende Arbeit mit Automatisierung und KI-Unterstützung verlässlich wird. Du lernst, Abläufe zu beschreiben, Schnittstellen zu nutzen, Antworten von Sprachmodellen zu prüfen und Agenten mit klaren Grenzen zu bauen.

Alle Codebeispiele sind ausgeführt und mit ihren Ausgaben abgedruckt. Die Praxisprojekte zeigen, wie du die Kontrollen von Anfang an einbaust, statt sie nachzurüsten.

Die weiteren Bände der Reihe KI PRAXIS: Band 1 KI im produktiven Alltag, Band 2 Recherche und Wissensmanagement, Band 4 lokale KI und eigene KI-Systeme, Band 5 Datenanalyse mit KI, Band 6 Dokumente, Präsentationen und Fachbücher, Band 7 Lernen mit KI und Band 8 Sicherheit und verantwortungsvolle Nutzung.

Eugen Becker | 1. Auflage, November 2026